MATH4065 – Lab Session 2

This session will introduce some of the basic ideas for programming in `R`. In particular, we will look at writing simple functions and using the `for`, `if` and `while` commands. These form the building blocks of writing more complicated programs.

Using scripts

For simple analysis, it is convenient to enter `R` commands directly into the Console window. However, for more complicated analysis we can write a series of commands in a script file and then ask `R` to read the commands. This makes it much easier to edit commands and also we can save the `R` script in case we want to repeat the analysis later.

Open a new script file (in RStudio) by going to File $\rightarrow$ New File $\rightarrow$ New Script. This opens a new script file in the Editor window. Type the following into the Editor window:

```
x<-rnorm(100)
hist(x)
```

We now save the script file by going to File $\rightarrow$ Save as ... and choosing a suitable file name, such as `myRscript.R`, for example.

If your script is already open in the Console window, you can run it by clicking on “Source”. If you just want to run part of a script, you can select the lines you want and click on “Run”.

If your srcipt is not open, you can either open it and proceed as above, or else go to Code $\rightarrow$ Source R code ... and select your script file. Alternatively, you can type in the command window

```
source("z:\\myRscript.R")
```

Note that the full filepath is necessary.

Once you run (or `source`) your script then `R` reads and executes the commands in the script file line-by-line. The histogram should have appeared in the Graphics window and you can view `x` by typing it in the Console window.

If we decide to simulate 1000 numbers, instead of 100, then we can simply edit the script, save it and then re-run it through `R`. Note that when you source the script, it reads in the last *saved* version of the script, so it is important that you save your script every time before running it. You can save it using the File menu or clicking the disk icon or pressing `Ctrl + S`.

Exercise: Edit the script file so that it samples $x_1, \ldots, x_{1000}$ from a $N(0, 9)$ distribution and plot a histogram of x^2.

#Writing functions

A function is a series of commands that can be called by using a simple name and arguments. We have already used some of `R`’s built-in functions, such as `rnorm` above. It is possible to write our own functions. Type the following into an `R` script.

```
plothistwithpdf<-function(n=100,mu=0,sigma=1){
   x<-rnorm(n,mean=mu,sd=sigma)
   hist(x,freq=FALSE)
   xval<-seq(min(x),max(x),length.out=100)
   pdf<-dnorm(xval,mean=mu,sd=sigma)
   lines(xval,pdf,col=``blue'')
   out<-list(xbar=0,s=0)
   out$xbar<-mean(x)
   out$s<-sd(x)
   out
}
```

Now read your script into `R` (e.g. by clicking on “Source”).

Note that `R` has not executed the function, but we have created a function in `R`’s memory called plothistwithpdf. Now if we type

```
ans<-plothistwithpdf(100,0,1)
```

in the Console window it will execute the function for our chosen values of {}, {} and {}. If we type

```
ans<-plothistwithpdf(1000,0,3)
```

then it will execute the function again, but this time sampling 1000 values from a $N(0, 9)$ distribution. Note that we can view the output from the function by typing

```
ans
```

Note that any function must have the following format:

```
function.name<-function(arguments){
   list of commands goes here...
}
```

Exercise: Write a function which takes a vector x as its only argument, produces a histogram of x and overlays the pdf of $N(\mu, \sigma^2/n)$ where μ and σ have to be estimated from the data. The function should return (output) the maximum likelihood estimates of μ and σ assuming x are random samples from a $N(\mu, \sigma^2)$ distribution.

###The `for` loop

If you have a statement (or statements) that you want to repeat a number of times, you can use the `for` statement to do so. Here is an example:

```
for (i in 1:10){
   x<-rnorm(100)
   xbar<-mean(x)
   print(xbar)
}
```

The `for` statement repeats the commands within the { } brackets a specified number of times. In the example above, the loop is repeated 10 times and within each iteration (or repetition) the sample mean of 100 observations from a standard normal distribution is printed on the screen.

Sometimes we may want the code within the loop to depend on which iteration is being executed. For example, if we want to construct a vector x of length 100 such that the ith entry is sampled from $N(i,1)$, then we could use the following `for` loop.

```
x<-rep(0,100)
for (i in 1:100){
   x[i]<-rnorm(1,i,1)
}
```

Note that it is important to define the variable `x` before attempting to assign values to the ith entry.

Exercise: Use the `for` command to construct a vector of length 100 where the ith entry in the vector is

$$\sum_{j=1}^{i} j.$$

The `if` statement

This is used when you want to do something if a condition is true and something else otherwise. The format of an `if` statement looks like this:

```
if ( condition ){
   commands to do if the condition is true...
}else{
   commands to do if the condition is false...
}
```

For example, suppose we want to generate a sample from a biased coin with probability $p = 0.3$ of heads. We can use the following commands:

```
u<-runif(1)
p<-0.3
if(u<p) {head<-1} else {head<-0}
print(head)
```

Note that it is not necessary to include the `else` statement if you want no commands to be executed if the condition is false.

In the above example we used the $<$ symbol to test if u was less than p. Other possible operators are listed in the table below.

Purpose	Operator
Equal to	==
Not equal to	!=
Less than or equal to	<=
Less than	<
Greater than or equal to	>=
Greater than	>
And	&

Purpose	Operator
Not	!=
Or	||

If we want to test if $p < x < q$ then we write in `R` : `(x>p & x<q)`

Exercise: Generate a random number, x, from $U(0,1)$. Use the `if` command to produce one roll of a six-sided dice using x.

Exercise: Generate a vector of 100 random uniform numbers from $U(0,1)$. Use the `for` and `if` commands to count the number of random numbers less than 0.5.

The `while` loop

If you need a loop for which you don't know in advance how many iterations there will be, you can use the `while` statement. In a `while` loop, a (list of) commands is carried out repeatedly while a condition is true. If something happens which makes the condition false, then the program exits the `while` loop. The format of a `while` loop looks like this:

```
while ( condition ){
   commands to repeat while the condition is true...
}
```

You construct conditions in the same way as for an `if` statement. Here is a simple example of a `while` loop.

```
x<-1
while (x<=100){
   print(x)
   x<-2*x
}
```

In the example above, the variable `x` is doubled in each iteration until `x` is greater than 100. In the following example, we sample from $U(0,1)$ until a random draw is greater than 0.9. The number of draws is then printed on the screen.

```
u<-0
t<-0
while (u<=0.9){
   t<-t+1
   u<-runif(1)
}
print(t)
```

Note that `for` and `while` loops are similar in that the commands within the loop are repeated. However, in a `for` loop the number of repetitions must be known in advance whereas in a `while` loop the repetitions keep occurring until the condition is satisfied. Be careful using a `while` loop because if the condition is never satisfied then the loop is executed indefinitely. You can use the ``stop'' icon to halt calculations if this occurs.

Exercise: Use the `while` command to generate random uniform numbers from $U(0,1)$ until the total sum of the numbers generated exceeds 50. How many random numbers did you need to generate?

Using functions

Our examples so far have been too short to break up into sub-programs. However, any sizeable `R` project will be easier to deal with if each specific task is put into its own sub-program or function, which your main program can call.

Here is another example of a function. This calculates $\sin(x)/x$. For $|x| >> 0$ this can be calculated directly using `R`'s built-in function sin(), but for values near 0, we would be calculating $0/0$, or something very close to it, so we use the power series:

$$\frac{\sin(x)}{x} = 1 - \frac{x^2}{3!} + \frac{x^4}{5!} + \ldots$$

Here's what we can do by defining a function:

```
sinc <- function(x){
   if (abs(x) > 1) {
      ## x not near 0: calculate in the obvious way.
      s <- sin(x)/x
   }
   else {
      ## x too close to 0: use power series instead.
      s <- 1
      term <- 1
      for( j in seq(3,100,by=2) ) {
         term <- term*(-x*x)/(j*(j-1))
         s <- s+term
         if(abs(term) < 1.e-10) break
      }
   }
   ## Return the value of s
   s
}
```

To help the user, this function contains comments which explain what the function is doing. Any line starting with the hash symbol is ignored by `R` and it is good practice to annotate your functions so others (and yourself) can understand them. This function also contains the `break` command to exit a `for` loop before all the iterations have been completed. In this `for` loop the index `j` takes the values $3, 5, 7, \ldots, 99$.

To use the function, put the text into a file and source it. Nothing appears to happen, but your function has been added to `R`'s memory. You can type

```
sinc(0.01)
```

and then the value of $\sin(0.01)/0.01$ will be printed out.

Exercise

Write a function which takes a vector x as its only argument and returns a list containing the mean and standard deviation of the values in x. Your function should do all the calculation itself, using loops and standard arithmetic operators. **It should not use** any of `R`'s built-in functions. (As an exception, you can use {} to get the number of elements in x.)

Use your function to calculate the mean and standard deviation of a set of numbers. Compare your results with those returned by `R`'s built-in functions `mean` and `sd`.

Simulation

In many examples and exercises we often refer to something like “let $(x_1, \ldots, x_n)$ be a random sample from a … distribution”. There are many techniques which allow us to draw a sample, in principle, from any distribution. Some of them are straightforward and some others are not. In this section we will only use `R` built-in functions to do this. For instance, the command {} enables us to simulate a random variable with any Normal distribution.

Why bother with simulation?

It is often the case that analytical solutions are not available. For instance, it is possible that a random variable X has pdf $f(x)$ such that $E[X] = \int_x xf(x)dx$ is not possible to derive explicitly, due to the fact that it is very hard to integrate the function $xf(x)$.

However, if we can simulate values of X, we could approximate $E[X]$ by simulating a large number of values and taking their mean.

The purpose of this session is to illustrate the cases where it is hard to derive results analytically, but using simulation is much easier.

`R` functions

Here are some of the `R` functions that can be used to sample values from common distributions.

Distribution	R function
Normal	rnorm(1, mu, sigma)
Beta	rbeta(1, alpha, beta)
Exp	rexp(1, lambda)
Gamma	rgamma(1, shape = lambda, rate = nu)
Poisson	rpois(1, lambda)
Chi Squared	rchisq(1, nu)
…	…

Look in the help files for other distributions, such as Student’s t distribution.

Exercises

Exercise 1

Use the commands above to simulate 10,000 values from a Gamma distribution and then draw a histogram to get a feel of that distribution.

Exercise 2

Use the commands above to simulate 10,000 values from a Normal distribution and then draw a Normal QQ-plot by making use of the command `qqplot` – Look in the help file too!

Exercise 3

Draw 10,000 samples from a Normal(0,1) distribution and from a student-t distribution with 3 degrees of freedom. Draw a QQ-plot to compare the two distributions. What do you see?

Exercise 4

Repeat the above but instead of 3 degrees of freedom use 100 and then draw a QQ-plot. What do you observe?

##A more interesting example

Suppose that $X \sim N(3,5)$ and $Y \sim Gamma(4,2)$ where the pdf of a $Gamma(a,b)$ distribution is parameterised as

$$f_X(x) = \frac{b^a}{\Gamma(a)} x^{a-1} \exp{(-bx)}, \quad x > 0 \quad a > 0, b > 0$$

Assume that X and Y are independent and denote $Z = X/Y$.

It is not straightforward to derive the probability density function of $f_Z(z)$ analytically (i.e. using formulae etc.) and therefore we will use simulation to compute $E[Z]$, $\mathrm{Var}[Z]$ etc.

One way to investigate the properties of $f_Z(z)$ is by simulating random samples (i.e i.i.d values) from the distribution with pdf $f_Z(z)$. How do we do that?

1. Simulate one random sample from $N(3,5)$ and one from $Gamma(4,2)$. Denote the values `x` and `y` respectively.
2. Compute `z = x/y` .
3. Repeat that a large number of times.

At the end we will have a large random sample from the distribution with pdf $f_Z(z)$. We can then use this sample to compute means, variances, quantiles etc, or any other quantity of interest.

Exercise

1. Compute the mean of Z.
2. Compute the variance of Z.
3. Find the distribution of $W = Z^2$.
4. Compute the probability $P(X < Y)$.

Optimisation

Suppose that we have some data $\boldsymbol{x} = (x_1, \ldots, x_n)$ and we assume that they are generated from a particular model, say M, which has some parameter(s) θ. We are interested in estimating the parameter(s) θ.

One way to estimate these parameter(s) is to maximize the likelihood function, i.e. to compute the MLEs. In the examples we've looked at in the problem classes so far, we could derive **analytically** what the MLEs were by finding the stationary points first and then checking whether or not these were maxima etc.

Nevertheless, it is often the case that such calculations are infeasible by hand, e.g. it is hard to find the MLEs by differentiating the log-likelihood, setting it equal to zero and solving for θ. Keeping in mind that the likelihood function, is just like any other ordinary function, one could look for the MLE numerically.

In other words, calculate the value of the (log) likelihood $L(\theta)$ for a series of values for θ and see which value of θ maximizes the likelihood. In this session, we will look at various ways on how to do this efficiently.

A simple example

Suppose that we have got a random sample $(x_1, \ldots, x_n)$ from an Exponential distribution with rate λ. It is easy to show that:

- log-likelihood: $l(\theta) = n\log(\theta) - \theta\sum_{i=1}^{n} x_i$;
- MLE: $\hat{\theta} = 1/\bar{x}$.

We will illustrate how someone could come up with the same answer using `R`.

First, we need to write a function in `R` which evaluates the likelihood for any dataset and any value θ:

```
log.lik.exp <- function(theta, x) {
   n <- length(x)  # find the number of observations
   sum.x <- sum(x) # compute the sum of x_i
   result <- n*log(theta) - theta*sum.x
   return(result)
}
```

Once we have that function, we can:

- compute the value of the log likelihood for a series of values of θ;
- plot the loglikelihood;
- see where it is maximised.

First, we need some data. Lets simulate some data using {}.

```
dataset <- rexp(100, 2.0)
```

In other words, in this case we know that the true value of the parameter θ is equal to 2. Hopefully, our program will return an estimate close to that value.

```
theta.values <- seq(0.0001, 6, len = 100);
log.lik.values <- rep(NA, len = length(theta.values))
for (i in 1:length(theta.values)) {
   log.lik.values[i] <- log.lik.exp(theta.values[i],dataset)
}
plot(theta.values, log.lik.values)
```

It seems that the is somewhere between 1 and 3, therefore:

```
theta.values <- seq(1, 3, len = 100);
log.lik.values <- rep(NA, len = length(theta.values))
for (i in 1:length(theta.values)) {
   log.lik.values[i] <- log.lik.exp(theta.values[i],dataset)
}
plot(theta.values, log.lik.values)
```

Use the command `which` to find the MLE:

```
index.MLE <- which(log.lik.values == max(log.lik.values))
theta.values[index.MLE]
```

Compare the answer above with

```
1/mean(dataset)
```

Another way: `optimise`

So far, we searched **manually** (ie brute-force) for the MLE. Alternatively, we could use an optimisation **algorithm** which searches for the minimum/maximum of a particular function over an interval.

Such algorithms often need as an input (or argument) the **function** and the interval in which they will look for the minimum/maximum.

Generally speaking, such algorithms are iterative which means that

- they start with some *initial value*, say θ_0 (often given by the user);
- *(numerical) derivatives* are computed to give us an idea about the direction to which the maximum/minimum is;
- repeat the above steps until *convergence*.

If the function for which we would like to find the minimum/maximum is one dimensional, then in {}, we can use the command:

```
optimise(function, interval, tolerance, ...)
```

By default, the command `optimise` finds the **minimum and not the maximum**. In our case, we are interested in finding MLEs (i.e. maxima). Therefore, there is tweak. We find the { minimum of the minus loglikelihood}, which obviously, is equivalent to finding the maximum of the loglikelihood.

Create the following function:

```
minus.log.lik.exp <- function(theta, x) {
   -log.lik.exp(theta,x)
}
```

Use the command now like this:

```
optimise(minus.log.lik.exp, c(0, 6), x = dataset)
```

{} will return something like this:

```
$minimum
[1] 2.382082

$objective
[1] 13.20269
```

which implies $\hat{\theta} = 2.382082$ and the value of the objective function (i.e. log-likelihood) at $\hat{\theta}$ is 13.20269.

Exercise

Suppose that we have some data from a Gamma distribution with parameters α and β. The density of a Gamma distribution is given as follows:

$$f_X(x) = \frac{\beta^\alpha}{\Gamma(\alpha)} x^{\alpha-1} e^{-\beta x}, \quad \alpha, \beta, x > 0.$$

1. Write a function which calculates the loglikelihood of a random sample $(x_1, \ldots, x_n)$.
2. Use that function to plot the loglikelihood (3d plot? contours?)
3. Use now the command `optim` (instead of `optimise`) to find the MLEs of α and β.